# Teaching Economics to the Machines

Hui Chen  Yuhan Cheng  Yanchu Liu  Ke Tang

MIT Sloan and NBER
School of Management, Shandong University
Lingnan College, Sun Yat-sen University
PBC School of Finance and Institute of Economics, Tsinghua University

December 2024

# Outline


1. Motivation
2. The Transfer Learning Framework
3. An application in option pricing
4. Conclusion

# The End of Theory?

- Chris Anderson, Wired (2008):

  *"All models are wrong, but some are useful." So proclaimed statistician George Box 30 years ago, and he was right. But what choice did we have? ... Today companies like Google, which have grown up in an era of massively abundant data, don't have to settle for wrong models. Indeed, they don't have to settle for models at all.*

- Ilya Sutskever, Reuters (2024):

  *The 2010s were the age of scaling, now we're back in the age of wonder and discovery once again ... Scaling the right thing matters more now than ever.*

# How to teach a robot to play pool?

# How should we teach a robot to play pool?

- Data-driven approach:
  - ↪ $x$: shot angle, position, force ...
  - ↪ $y$: ball direction, speed, spin ...
  - ↪ However, labeled data are often expensive to collect.
- Theory-driven approach:
  - ↪ Model of elastic collisions
  - ↪ Conservation of linear momentum and kinetic energy ⇒ complete predictions for ball movements
  - ↪ However, the model is likely misspecified (inelastic collision, imperfect surface ...)
- Why not combine the knowledge from theory and data?
  - ↪ Our answer: transfer learning

# Overview of results

- Transfer learning (TL): First learn from theory (through synthetic data), then from real data (with fine-tuning).
- In an application to option pricing, TL significantly outperforms a conventional deep learning (DL) model and a popular structural model.
- This is especially true:
  - ↪ for median and long time-to-maturity, OTM, liquid options;
  - ↪ when the market condition is volatile; or
  - ↪ when the input feature appears "uncommon" relative to the training sample.
- TL is more stable than DL. Its advantage over DL is more pronounced in smaller samples.
- TL can help identify features missing from the structural model.

# Related Literature

- Economic restrictions in ML: Garcia and Gencay (2000), Feng et al. (2023), Chen, Pelger, and Zhu (2024), Bryzgalova, Pelger, and Zhu (2023), and Bryzgalova et al. (2024)
- Physics-informed neural networks: Raissi, Perdikaris, and Karniadakis (2019)
- Bayesian VAR: Doan, Litterman, and Sims (1984), Litterman (1986), DeJong, Ingram, and Whiteman (1993), Ingram and Whiteman (1994), Del Negro and Schorfheide (2004)
- Boosting: Almeida, Fan, Freire, and Tang (2023)
- ML applications in economics and finance:
  - ↪ Early contributions: Hutchinson, Lo, and Poggio (1994), Chen and White (1999), Chen and Ludvigson (2009)
  - ↪ Linear models: Kelly and Pruitt (2013), Chinco, Clark-Joseph, and Ye (2019), Kozak, Nagel, and Santosh (2023)
  - ↪ Nonlinear model: Gu, Kelly, and Xiu, (2020); Freyberger, Neuhierl, and Weber (2020), Bianchi, Buchner, and Tamoni (2021), Cong et al. (2022), Bali et al. (2023), Chen, Pelger, and Zhu (2024)

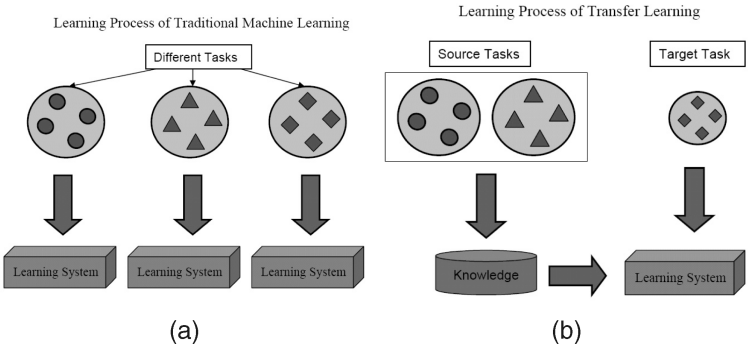
# Outline


1. Motivation
2. The Transfer Learning Framework
3. An application in option pricing
4. Conclusion

# Transfer learning

- Transfer Learning: Using pre-trained models on large datasets (source domain) to accelerate learning on a smaller dataset (target domain) for a specific task.
  - ↪ computer vision: pre-train CNNs on ImageNet ⇒ medical image analysis
  - ↪ natural language processing: pre-trained LLMs ⇒ sentiment analysis

- In our setting: Source domain = economic model; target domain = empirical data.



# Teaching economics to the machines

- The neural networks trained on the simulated data from a theoretical model inherit the structural restrictions from the theory.
- TL: Use theory-implied parameter values as the starting point. Fine-tuning allows the structural restrictions to regulate the learning on real data.
- Why might this be helpful?
  - ↪ Bias-variance tradeoff: Valid restrictions help with learning. Misspecified models also add bias.
  - ↪ Theory-informed initialization can mitigate issues in numerical optimization, such as avoiding local minima and saddle points.
- Potential benefits:
  - ↪ Variance reduction
  - ↪ Speed up training on real data; less demand for real data
  - ↪ Improve generalizability beyond training data boundaries

# The setup

- Denote by $\mathscr{X}$ and $\mathscr{Y}$ the input and target space, respectively, with unknown probability distribution $\mathbb{P}_{(\mathscr{X},\mathscr{Y})}$.
- Denote a neural network with $L$ layers as $F(L;\sigma_1,\cdots,\sigma_L;W_1,\cdots,W_L)$.
  - $\hookrightarrow$ $\sigma_i$: activation function; $W=[W_1,\cdots,W_L]$: network weights
- Standard approach to train a neural network: Find a training set $S=((x_i,y_i))_{i=1}^m$ and a loss function $\mathscr{L}(f(x),y)$. Then minimize the empirical loss:

$$W^* = \arg\min_W \frac{1}{m}\sum_{i=1}^m \mathscr{L}(F(x_i;W),y_i)$$

# The setup

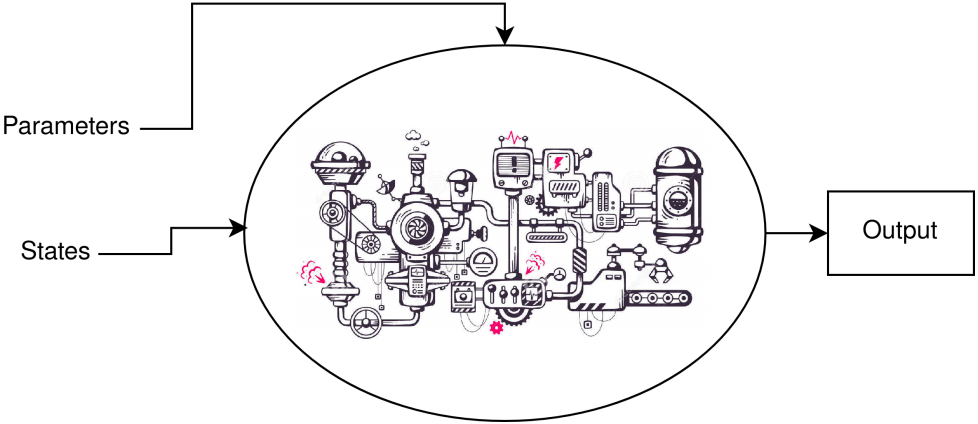
- Economic model: Could lead to a fully specified joint distribution for $x$ and $y$ as $\mathbb{Q}_{(\mathscr{X},\mathscr{Y})}$. Or, it could just be the conditional expectation of the target $y$,

$$\mathbb{E}^{\mathbb{Q}}[y|x] = g(x)$$

- A model is typically designed to be parsimonious, with a small number of states, some of which can be latent. There are also model parameters.

- The model can be misspecified: $\mathbb{Q} \neq \mathbb{P}$

# Model as a set of restrictions




$$g\,(\underbrace{s, h \mid \theta}_{x}) = \mathbb{E}^{\mathbb{Q}}[y|x]$$

# Source domain

- Generate synthetic data $\left((\tilde{x}_i, \tilde{y}_i)\right)_{i=1}^{M}$ from the model-implied distribution, $\mathbb{Q}_{(\mathscr{X},\hat{\mathscr{Y}})}$.

$$\tilde{y}_i = g(\tilde{x}_i)$$

- In the source domain, we train the neural networks to learn from the synthetic dataset:

$$\widetilde{W}^* = \underset{\widetilde{W}}{\arg\min}\left(\lambda_0 \mathscr{L}_0 + \sum_{j=1}^{K} \lambda_j \mathscr{L}_j\right)$$

- Economics-informed losses:

$$\mathscr{L}_0 = \sum_{i=1}^{M} w_{0i} \left| F(\tilde{x}_i | \widetilde{W}) - \tilde{y}_i \right|$$

$$\mathscr{L}_j = \sum_{i=1}^{M} w_{ji} \left| \partial_j F(\tilde{x}_i | \widetilde{W}) - \partial_j g(\tilde{x}_i) \right| \quad \forall j > 0$$

$\hookrightarrow$ Physics-informed losses: Raissi, Perdikaris, and Karniadakis (2019)

# Benefits of neural networks

Chen, Didisheim and Scheidegger (2023)

- Knowledge of true DGP ⇒ Generate as much synthetic data as desired (can be afforded) with high information-noise ratio.
  - ↪ The risks of over-fitting are low.
- Expressivity: Universal approximation theorem for shallow and deep networks.
  - ↪ Hornik, Stinchcombe, and White (1989), Hanin and Sellke (2017), Hanin (2019)
- Curse of dimensionality: With suitable target function and activation function, one can train accurate surrogate with sample sizes growing polynomially in model input dimension.
  - ↪ Berner, Grohs, and Jentzen (2020), Grohs et al. (2023)

# Target domain

- In the target domain, train with real data $\big((x_i, y_i)\big)_{i=1}^{n}$.
- We might want to add many features beyond those in a structural model.
  - ↪ In the source domain, supplement synthetic data with random draws of additional features.
- How to deal with hidden states and structural parameters, which are not directly observable?
  - ↪ Filtering or conditioning down.
  - ↪ The network trained in the source domain can be used to do filtering efficiently (Chen, Didisheim and Scheidegger 2023).

# Target domain

- Identical network architecture as in the source domain; use the weights from the source domain for initialization:

$$\widehat{W}^* = \arg\min_{\widehat{W}} \sum_{i=1}^{m} w_i \left| F(x_i|\widehat{W}) - y_i \right|,$$

  with initial weights $\widetilde{W}^*$.

- Fine-tuning: Small learning rate and early stopping.

- Alternative approaches:
  - ↪ Augment the network from the source domain with new layers.
  - ↪ Partial fine-tuning: fixing bottom part of the network (frozen layers)

# Comparison with alternative approaches

- Imposing structural restrictions in ML model:
  - ↪ Long tradition in rational expectations econometrics
  - ↪ Examples: Physics-informed Neural Networks (Raissi, Perdikaris, and Karniadakis 2019); Using NA conditions in SDF estimation (Chen, Pelger, and Zhu 2023)
  - ↪ In TL, structural restrictions help with learning but are not treated as hard constraints, since misspecified restrictions likely add more bias.

- Bayesian approach:
  - ↪ Bayesian VAR: Deriving informative priors from an economic model to aid VAR estimation.
  - ↪ Examples: Random walk (Doan, Litterman, and Sims 1984); DSGE (DeJong, Ingram, and Whiteman 1993; Del Negro and Schorfheide 2004)
  - ↪ TL: easier to implement, significantly more flexible at capturing nonlinearity, and controls the "strength of prior" differently.

- Boosting:
  - ↪ Train an ML model on the prediction errors from a structural model.
  - ↪ Example: Boosting parametric option pricing models (Almeida, Fan, Freire, and Tang 2023)
  - ↪ The structural model restrictions are not used to guide the learning in the boosting step.

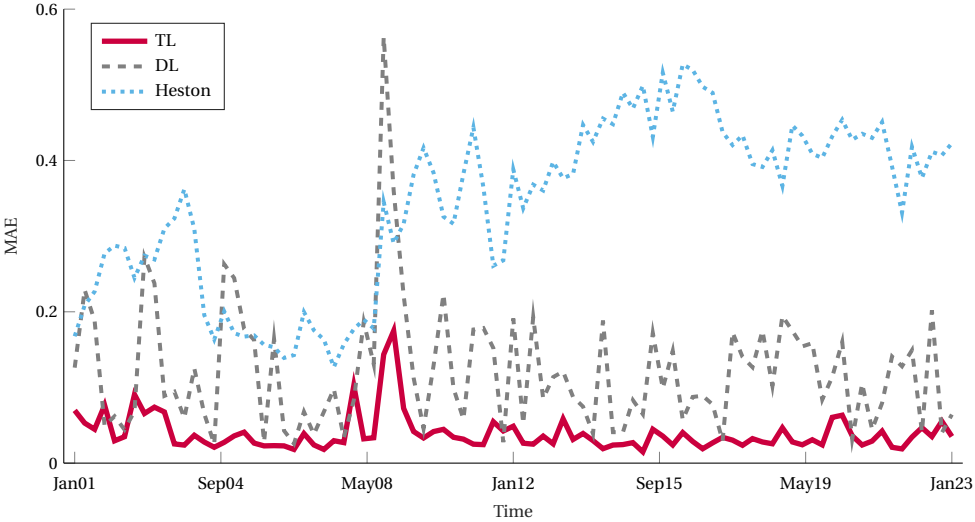
# Outline


1. Motivation
2. The Transfer Learning Framework
3. An application in option pricing
4. Conclusion

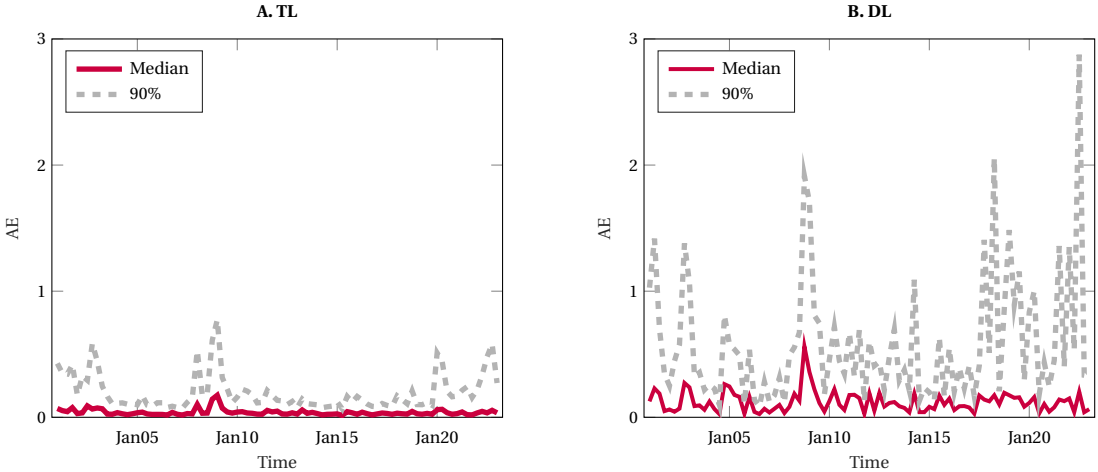
# Example application

- A horse race between theory-based transfer learning (TL) and standard deep neural networks (DL) for option pricing.
  - ↪ Use the Black-Scholes model in the source domain.
  - ↪ Augment the set of features with past returns, bid-ask spreads, put-call ratios ... (total of $d = 16$ features)
- Architecture: 16 hidden-layer ResNet, 22 neurons in each layer ($> 7,800$ trainable parameters), LeakyRelu activation
- Rolling window: At the end of each quarter, use data from the past 3 months to train the models; use the models to predict option prices in the next 3 months.
  - ↪ Mimicking a setting with limited labeled data in the target domain.
- Other candidate models: Heston model, DL with BS penalty, boosted BS

# TL outperforms DL and Heston out of sample (based on IV-MAE)



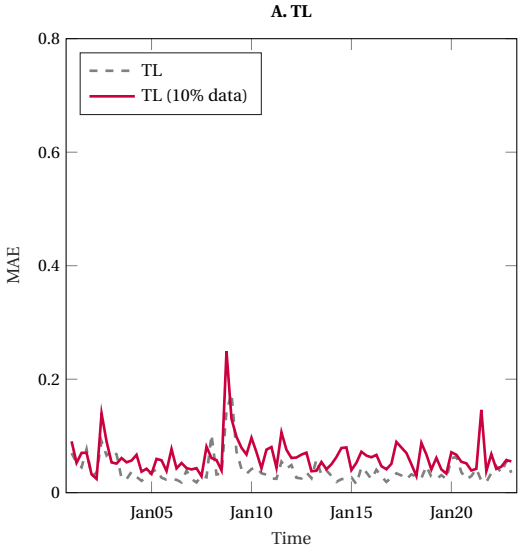
# TL has less extreme outliers

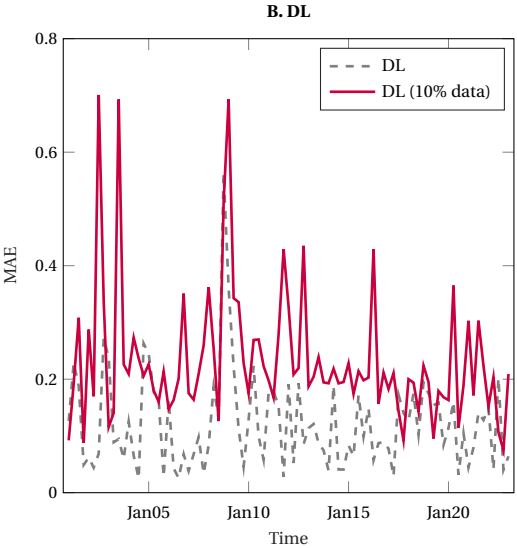


# Performance attribution

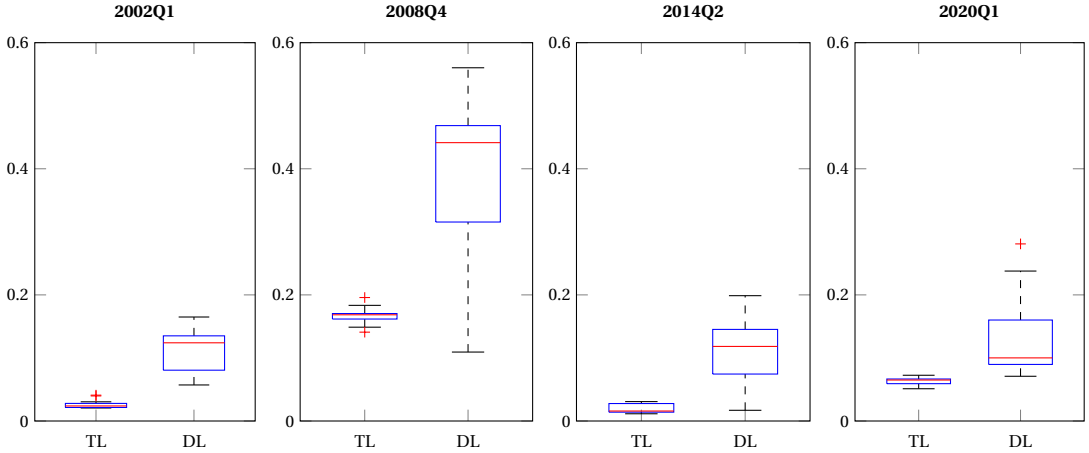
- Regressing the difference in absolute pricing errors between the DL and TL model at contract-day level on a variety of attributes.
- TL has a bigger advantage relative to DL:
  - ↪ for options with median and long maturity (>14 days), OTM (especially DOTM), with better liquidity (high volume);
  - ↪ when the market is more volatile;
  - ↪ when volatility spikes;
  - ↪ when test sample is "uncommon" relative to training sample (Mahalanobis distance).

# TL is better at learning from a smaller sample



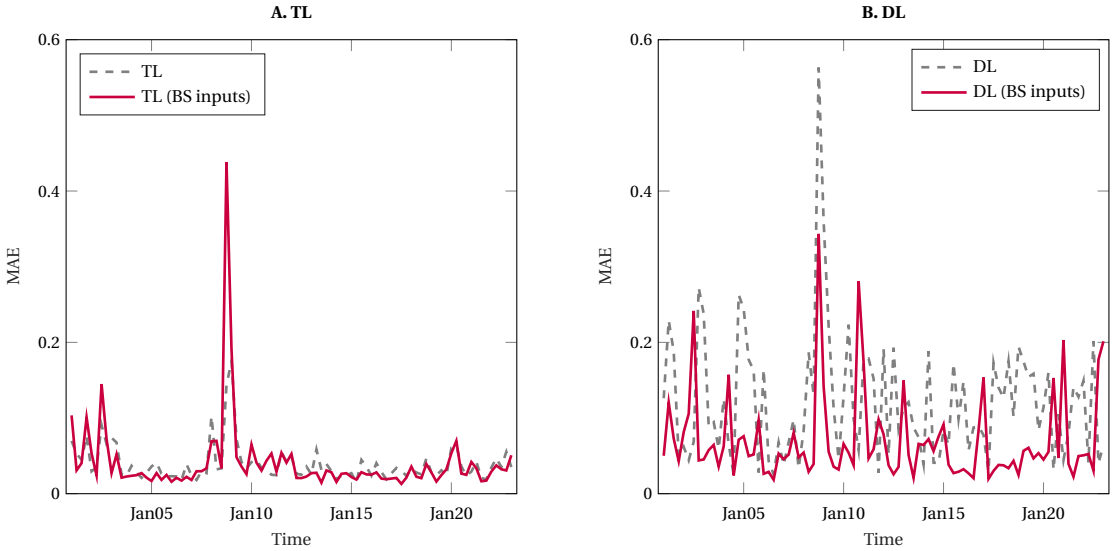
# TL is less sensitive to the randomness of training



# What is source of TL out-performance?

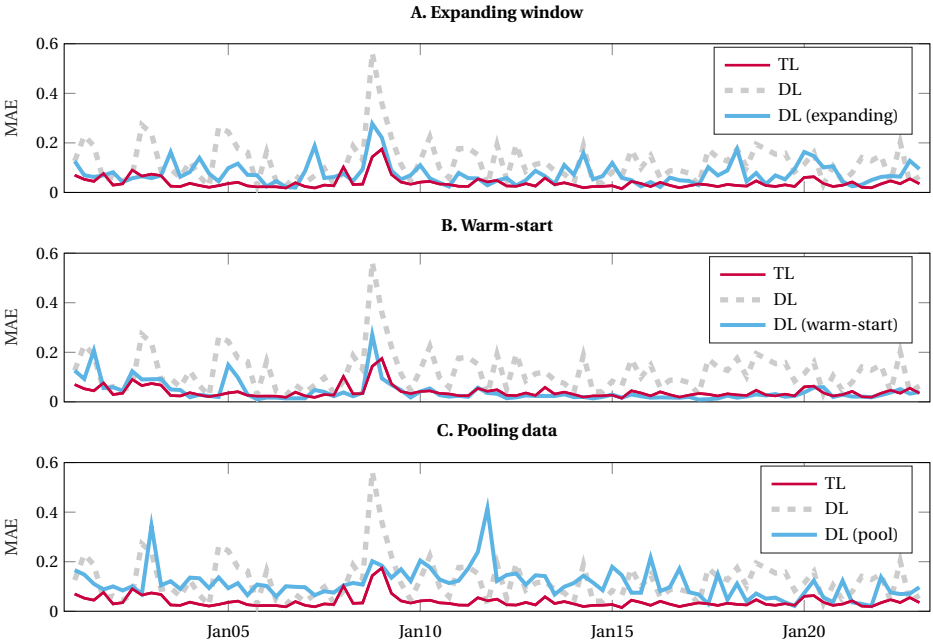
- Theory helps identify relevant inputs?
  - ↪ Compare to DL with the same inputs
- More data?
  - ↪ Rolling-window vs. expanding-window
  - ↪ Pooling real data with simulated data
- Other ways to bring in structural model information
  - ↪ Introducing a penalty in the loss function based on the violation of BS restrictions.
  - ↪ Boosting BS with OLS.
- Would any restrictions work?
  - ↪ Use no-arbitrage restrictions in the source domain

# Theory does more than identifying inputs



# What is source of TL out-performance?

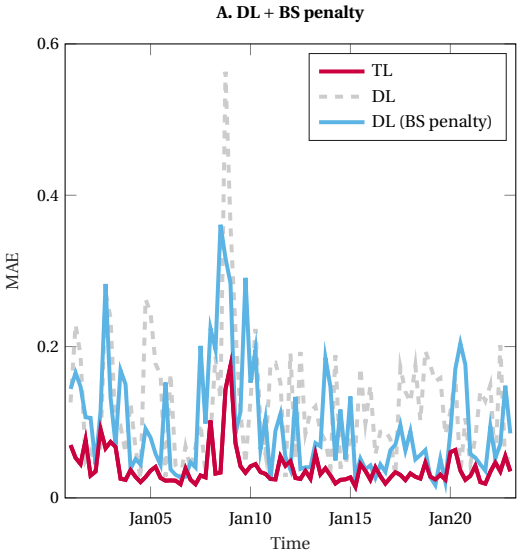
- Theory helps identify relevant inputs?
  - ↪ Compare to DL with the same inputs
- More data?
  - ↪ Rolling-window vs. expanding-window
  - ↪ Pooling real data with simulated data
- Other ways to bring in structural model information
  - ↪ Introducing a penalty in the loss function based on the violation of BS restrictions.
  - ↪ Boosting BS with OLS.
- Would any restrictions work?
  - ↪ Use no-arbitrage restrictions in the source domain

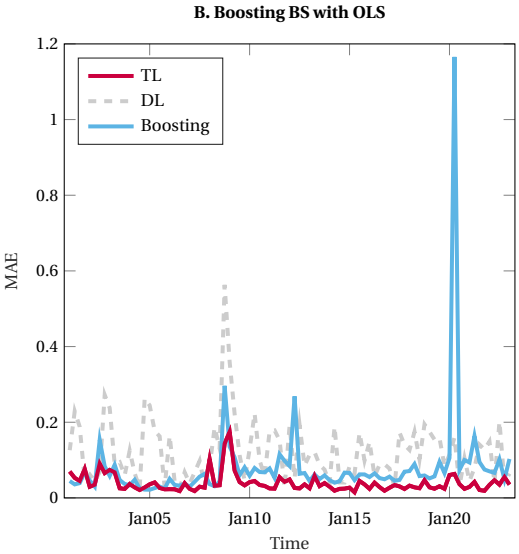

A. Expanding window
MAE
0.6
0.4
0.2
0
TL
DL
DL (expanding)
B. Warm-start
MAE
0.6
0.4
0.2
0
TL
DL
DL (warm-start)
C. Pooling data
MAE
0.6
0.4
0.2
0
TL
DL
DL (pool)
Jan05
Jan10
Jan15
Jan20

# What is source of TL out-performance?

- Theory helps identify relevant inputs?
  - ↪ Compare to DL with the same inputs
- More data?
  - ↪ Rolling-window vs. expanding-window
  - ↪ Pooling real data with simulated data
- Other ways to bring in structural model information
  - ↪ Introducing a penalty in the loss function based on the violation of BS restrictions.
  - ↪ Boosting BS with OLS.
- Would any restrictions work?
  - ↪ Use no-arbitrage restrictions in the source domain

# Alternative ways to incorporate model information

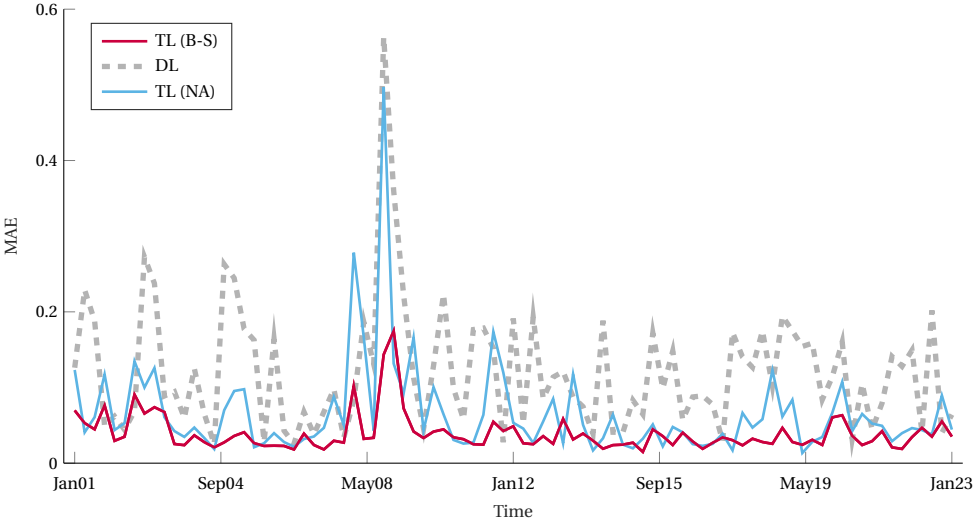


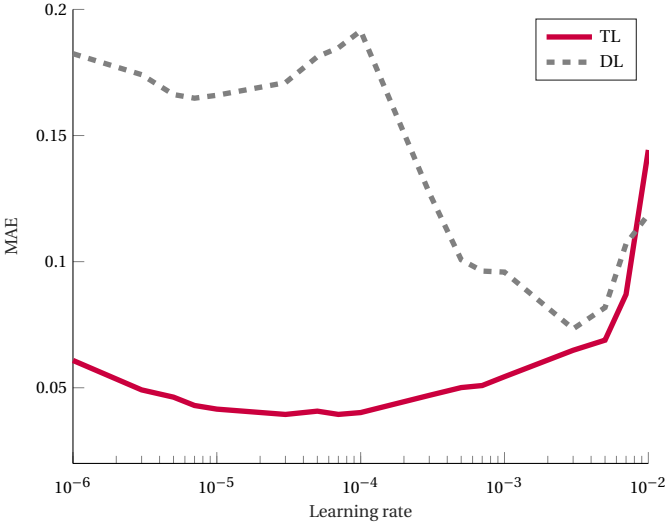
# What is source of TL out-performance?

- Theory helps identify relevant inputs?
  - ↪ Compare to DL with the same inputs
- More data?
  - ↪ Rolling-window vs. expanding-window
  - ↪ Pooling real data with simulated data
- Regularization by imposing structural restrictions directly?
  - ↪ Introducing a penalty in the loss function based on the violation of the structural model restrictions.
  - ↪ Boosting the structural model with OLS.
- Would any restrictions work?
  - ↪ Use no-arbitrage restrictions in the source domain

# Using no-arbitrage restrictions in the source domain



# The importance of fine-tuning

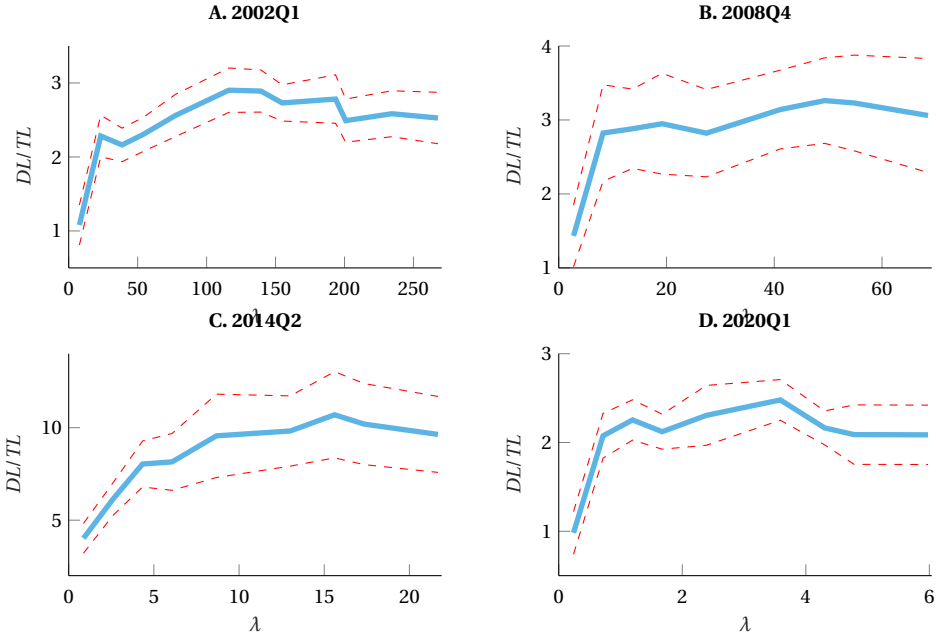


# TL ≠ Informative Prior

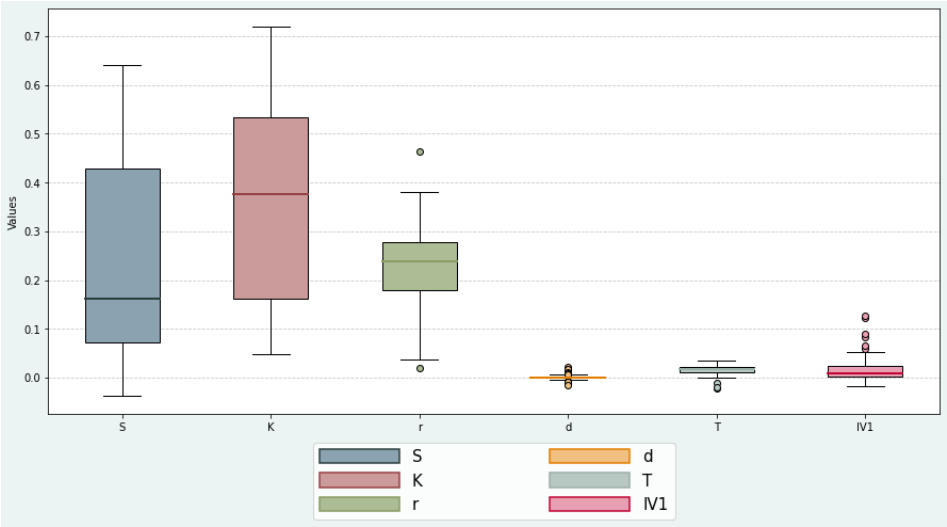
- Consider relative sample size

$$\lambda = \frac{\text{sample size of synthetic data}}{\text{sample size of real data}}$$

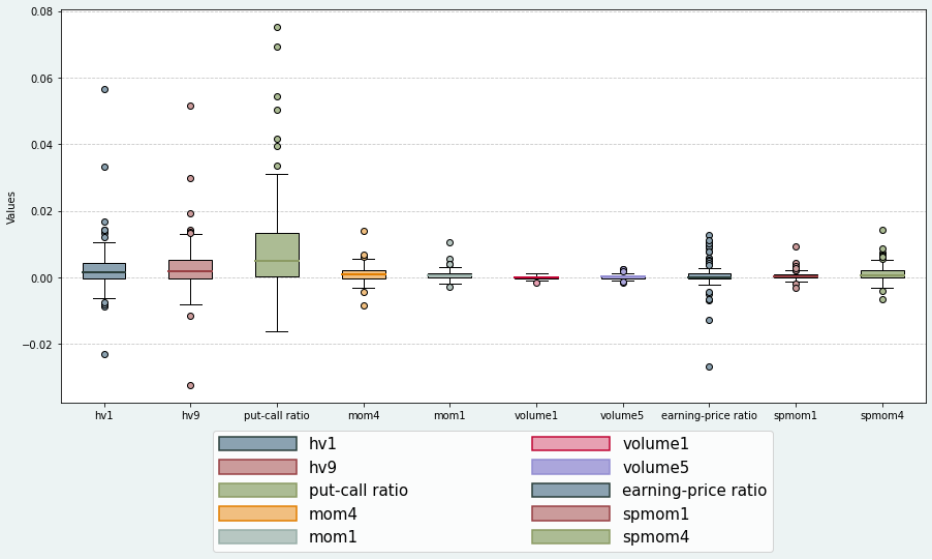
- In Bayesian VAR, $\lambda$ effectively controls the prior precision.
- Del Negro and Schorfheide (2004): An inverse U-shaped relation between forecasting accuracy and $\lambda$.
  - ↪ Intuition: Too little or too much weight on the DSGE model can both hurt performance.
- TL: More synthetic data will improve the accuracy of the structural model. But the learned weights in the source domain are still only the initial values for the next stage.
- How much the final weights will devaite depends on:
  - ↪ Gap between the structural model and true DGP
  - ↪ Hyperparameters of learning in the target domain: learning rate, epochs


A. 2002Q1
B. 2008Q4
C. 2014Q2
D. 2020Q1
DL/TL
λ
0
50
100
150
200
250
20
40
60
5
10
15
2
4
6
1
2
3
4

# Feature importance



# Feature importance



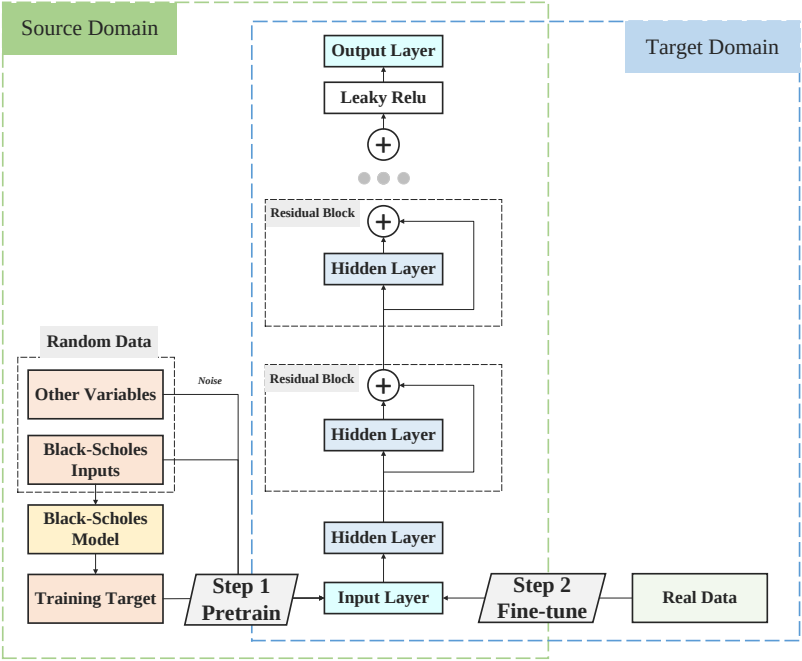
# Outline


1. Motivation
2. The Transfer Learning Framework
3. An application in option pricing
4. Conclusion

# Conclusion

- Guiding ML with economic theory through transfer learning.
  - ↪ Higher accuracy and stability, especially in smaller samples.
  - ↪ Generalizability. Dealing with structural breaks.
- Not specific to neural networks. The framework easily applies to other ML models.
  - ↪ More useful when real data are scarce + but still need flexibility
- Work in progress:
  - ↪ macro forecasting
  - ↪ prepayment model
  - ↪ credit risk model
  - ↪ portfolio construction
  - ↪ ...

Theories are dead? Long live theories!


Source Domain
Target Domain
Output Layer
Leaky Relu
Residual Block
Hidden Layer
Residual Block
Hidden Layer
Hidden Layer
Input Layer
Random Data
Other Variables
Black-Scholes Inputs
Noise
Black-Scholes Model
Training Target
Step 1 Pretrain
Step 2 Fine-tune
Real Data